

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Hoàng Hữu Đức	23020046
Nguyễn Nho Dương	23020034
Lê Minh Tuấn	23020149
Phạm Minh Thông	23020164

**CÔNG CỤ KIỂM THỬ
CHECKSTYLE**

BÁO CÁO BÀI TẬP LỚN

Bộ môn: Kiểm thử và đảm bảo chất lượng phần mềm

HÀ NỘI - 2025

LỜI CAM ĐOAN

Chúng em xin cam đoan: Báo cáo này là kết quả của công việc nghiên cứu và thực hiện của nhóm. Những gì chúng em viết ra hoàn toàn trung thực, chính xác và không có sự sao chép từ các tài liệu, không sử dụng kết quả của người khác mà không trích dẫn cụ thể. Các dữ liệu, kết quả thực nghiệm, và phân tích được trình bày trong báo cáo đều là chính xác và thực tế, không được làm giả hoặc bao gồm bất kỳ thông tin sai lệch nào. Các tài liệu tham khảo được liệt kê đầy đủ và chính xác trong phần “tài liệu tham khảo” của báo cáo. Nếu vi phạm những điều trên, chúng em xin hoàn toàn chịu trách nhiệm về những hậu quả pháp lý liên quan theo quy định của Trường Đại học Công nghệ - ĐHQGHN.

Hà Nội, ngày 14 tháng 12 năm 2025

Sinh viên

Hoàng Hữu Đức

MÔ TẢ ĐÓNG GÓP

Họ và tên	Mã sinh viên	Công việc thực hiện	Mức độ đóng góp
Hoàng Hữu Đức	23020046		
Nguyễn Nho Dương	23020034		
Phạm Minh Thông	23020164		
Lê Minh Tuấn	23020149		

MỤC LỤC

Lời cam đoan	i
Mô tả đóng góp	ii
Mục lục	iii
Danh mục hình ảnh	v
Danh mục bảng	vi
Chương 1. Giới thiệu	1
1.1. Bối cảnh	1
1.2. Lý do chọn đề tài	1
1.3. Cấu trúc của báo cáo	2
Chương 2. Tổng quan về Checkstyle	3
2.1. Giới thiệu chung	3
2.2. Kiến trúc	4
2.2.1. Cây cấu hình	4
2.2.2. Cây cú pháp	5
2.2.3. Checker	6
2.2.4. FileSetCheck	6
2.2.5. Filter Systems	7
2.2.6. AuditListener	7
2.3. Nguyên tắc hoạt động	8
2.3.1. Tổng quan luồng hoạt động	8
2.3.2. Chi tiết quá trình thực thi	9
Chương 3. Áp dụng Checkstyle phân tích dự án thực tế	12
3.1. Lựa chọn dự án và thiết lập môi trường phân tích	12

3.1.1. Lựa chọn dự án	12
3.1.2. Thiết lập môi trường	12
3.2. Thực thi kiểm thử	13
3.3. Kết quả kiểm thử	14
3.4. Phân tích kết quả	16
3.5. Đề xuất khắc phục	17
Chương 4. So sánh	20
4.1. Đánh giá	20
4.2. Kết luận	22
Tài liệu tham khảo	23

DANH MỤC HÌNH ẢNH

Hình 2.1	Luồng hoạt động của Checkstyle	8
Hình 3.1	Tổng quan các lỗi vi phạm trong dự án MegaBasterd	16

DANH MỤC BẢNG

Bảng 4.1 So sánh Checkstyle và SonarQube	21
--	----

Chương 1

Giới thiệu

1.1. Bối cảnh

Trong thời đại phát triển nhanh chóng của công nghệ thông tin, chất lượng phần mềm là một yếu tố quyết định đến sự thành công của các dự án phát triển. Tuy nhiên, việc duy trì tiêu chuẩn mã hóa và đảm bảo mã nguồn tuân thủ các quy tắc thiết lập là một thách thức lớn, đặc biệt khi các đội phát triển có nhiều thành viên với các thói quen lập trình khác nhau.

Ngôn ngữ lập trình Java là một trong những ngôn ngữ phổ biến nhất hiện nay, được sử dụng rộng rãi trong phát triển các ứng dụng doanh nghiệp, hệ thống Backend, và các dự án mã nguồn mở quy mô lớn. Tuy nhiên, Java cũng là ngôn ngữ có tính linh hoạt cao, cho phép các lập trình viên viết code theo nhiều cách khác nhau. Điều này dẫn đến:

- Mã nguồn thiếu tính nhất quán trong định dạng và cấu trúc.
- Khó khăn trong việc code review và bảo trì.
- Rủi ro cao trong việc phát sinh các lỗi tiềm ẩn liên quan đến định dạng và quy ước đặt tên.
- Tăng chi phí bảo trì và cập nhật phần mềm.

Để giải quyết những vấn đề trên, các công cụ phân tích mã tĩnh (static code analysis tools) như Checkstyle, SpotBugs, PMD, v.v. đã được phát triển. Trong đó, **Checkstyle** là một công cụ chuyên biệt dành riêng cho ngôn ngữ Java, giúp lập trình viên tự động hóa quá trình kiểm tra và thực thi các tiêu chuẩn mã hóa.

1.2. Lý do chọn đề tài

Nhóm lựa chọn Checkstyle làm chủ đề của báo cáo vì công cụ này giúp phát hiện sớm các vi phạm về tiêu chuẩn mã hóa ngay trong quá trình phát triển,

từ đó duy trì tính nhất quán của mã nguồn trước khi được tích hợp vào nhánh chính của dự án.

Bên cạnh đó, Checkstyle có khả năng tích hợp dễ dàng với các môi trường phát triển phổ biến như IntelliJ IDEA, Eclipse cũng như các hệ thống CI/CD như Jenkins, GitLab CI hay GitHub Actions, cho phép tự động hóa việc kiểm tra mã nguồn trong quá trình lập trình và review code.

Một lý do quan trọng khác là Checkstyle là công cụ miễn phí và mã nguồn mở, cho phép người dùng sử dụng, nghiên cứu và mở rộng mà không bị ràng buộc về chi phí hay bản quyền, đồng thời giúp người dùng dễ dàng tìm hiểu và nắm bắt cách thức hoạt động của công cụ.

Ngoài ra, Checkstyle cho phép tùy chỉnh linh hoạt các quy tắc kiểm tra thông qua file cấu hình XML, giúp công cụ thích nghi với các tiêu chuẩn mã hóa khác nhau của từng tổ chức hoặc dự án.

Nhờ những đặc điểm trên, Checkstyle góp phần nâng cao chất lượng tổng thể của mã nguồn, giảm bớt công sức kiểm tra thủ công và cải thiện khả năng bảo trì của phần mềm trong các dự án thực tế.

1.3. Cấu trúc của báo cáo

Phần còn lại của báo cáo này được trình bày như sau:

- [Chương 2](#): Tìm hiểu chi tiết về công cụ Checkstyle, bao gồm kiến trúc, nguyên tắc hoạt động, và các tính năng chính.
- [Chương 3](#): Áp dụng Checkstyle vào phân tích một dự án Java thực tế và đưa ra các đề xuất khắc phục.
- [Chương 4](#): Đánh giá, kết luận.

Chương 2

Tổng quan về Checkstyle

2.1. Giới thiệu chung

Checkstyle là một công cụ phân tích mã nguồn tĩnh, được phát triển và thiết kế chuyên biệt cho ngôn ngữ Java. Công cụ này hỗ trợ lập trình viên Java tuân thủ các tiêu chuẩn mã nguồn nhất định, góp phần nâng cao chất lượng và tính chuyên nghiệp của dự án. Checkstyle có khả năng hoạt động trên nhiều nền tảng như Windows, macOS và Linux/Unix, đồng thời được phát triển hoàn toàn bằng ngôn ngữ Java.

Mục đích chính của Checkstyle là kiểm tra mã nguồn phù hợp với các bộ quy tắc một cách tự động, từ đó giảm thiểu công sức kiểm tra thủ công và hạn chế các lỗi do con người gây ra. Thông qua việc đảm bảo tính nhất quán trong mã nguồn, Checkstyle giúp cải thiện khả năng đọc, bảo trì và tái sử dụng code, đồng thời phát hiện sớm các vi phạm liên quan đến định dạng, cấu trúc và thiết kế ngay trong quá trình phát triển dự án.

Bên cạnh đó, Checkstyle còn hỗ trợ nhiều tính năng hữu ích như tích hợp vào quy trình CI/CD để tạo báo cáo vi phạm chi tiết và kết hợp với các công cụ như Jenkins, Maven hay Gradle. Công cụ cho phép kiểm tra mã nguồn dựa trên các quy tắc được cấu hình sẵn (Google Java Style hoặc Sun Code Conventions) hoặc các quy tắc tùy chỉnh theo nhu cầu của dự án. Ngoài ra, Checkstyle có thể được tích hợp dưới dạng plugin trong các IDE phổ biến như Eclipse, IntelliJ IDEA và NetBeans, giúp lập trình viên phát hiện vi phạm ngay trong quá trình lập trình.

Như vậy, nhờ tính tiện dụng, khả năng tích hợp linh hoạt và việc được phát triển dưới dạng dự án mã nguồn mở, Checkstyle được xem là một lựa chọn khá tốt và phù hợp cho nhiều dự án phần mềm Java hiện nay.

2.2. Kiến trúc

Kiến trúc của Checkstyle được thiết kế dựa trên sự kết hợp giữa mẫu thiết kế *Pipe and Filter* và cơ chế *Event-Driven*. Hệ thống mang tính module hóa cao, cho phép mở rộng linh hoạt thông qua việc cấu hình thay vì phải biên dịch lại mã nguồn. Dưới đây là các thành phần cốt lõi tạo nên bộ khung của Checkstyle:

2.2.1. Cây cấu hình

Một cấu hình (Configuration) của Checkstyle quy định những module nào sẽ được sử dụng và áp dụng lên các tệp mã nguồn Java. Các module này được tổ chức theo dạng cây, trong đó gốc của cây luôn là module Checker. Dưới Checker là các nhóm module chính:

- File Set Checks: Các module sử dụng để kiểm tra các vi phạm từ các file đầu vào.
- Filters: Các module dùng để lọc các sự kiện kiểm tra (audit events), từ đó quyết định các yêu cầu có được chấp nhận hay bị loại bỏ.
- Audit Listeners: Các module nhận và báo cáo các sự kiện sau khi đã được kiểm duyệt.

Khi sử dụng Checkstyle để tiến hành kiểm thử, người dùng cần chỉ định một tập tin XML, trong đó các phần tử thể hiện cấu trúc cây module của cấu hình, các module được bật, và các thuộc tính được thiết lập cho từng module.

```
<module name="Checker">
  <module name="JavadocPackage"/>
  <module name="TreeWalker">
    <property name="tabWidth" value="4"/>
    <module name="AvoidStarImport"/>
    <module name="ConstantName"/>
    ...
  </module>
</module>
```

Chương trình 2.1 — Ví dụ về cấu hình Checkstyle

Một module trong file cấu hình XML được biểu diễn bằng thẻ `<module>` cùng trường `name`. Với mỗi module, Checkstyle sẽ load các class được chỉ định trong trường `name` của các thẻ `<module>`, theo các nguyên tắc sau:

- Load trực tiếp một class theo tên đầy đủ của nó (ví dụ `com.puppycrawl.tools.checkstyle.TreeWalker`), sử dụng khi ta cần tích hợp các module bên thứ ba.
- Load một class được Checkstyle quy định sẵn (ví dụ, khi ta muốn load class `com.puppycrawl.tools.checkstyle.checks.AvoidStarImport`, ta chỉ cần chỉ định trường `name` là `AvoidStarImport`, khi đó công cụ sẽ tự động tìm class theo các package `com.puppycrawl.tools.checkstyle`, `com.puppycrawl.tools.checkstyle.filters`, `com.puppycrawl.tools.checkstyle.checks` và các subpackage khác của các package này trong Checkstyle distribution)
- Áp dụng các quy tắc trên cho tên module sau khi thêm hậu tố “Check” (ví dụ tải lớp `com.puppycrawl.tools.checkstyle.checks.ConstantNameCheck` cho phần tử `<module name="ConstantName"/>`)

Để điều chỉnh các hành vi của một module, ta có thể dùng thẻ `<properties>` với các trường `name` và `value`. Ví dụ, trong đoạn code trên, ta quy định độ rộng của một kí tự tab là 4 cho module `TreeWalker` cùng tất cả các module con của nó.

2.2.2. Cây cú pháp

Để kiểm tra mã nguồn, hầu hết các tiêu chí lựa chọn sử dụng cây cú pháp (AST - Abstract Syntax Tree) được dựng trực tiếp từ mã nguồn Java gốc. Một nút trên cây cú pháp được biểu diễn bằng lớp `DetailAST`, tương ứng với một thành phần cú pháp cụ thể trong mã nguồn Java, ví dụ như class, method, biến, biểu thức... Cấu trúc của một nút trên cây có dạng:

- `getType()`: Loại của nút, ví dụ `CLASS_DEF` thể hiện định nghĩa của một class, hay `METHOD_DEF` thể hiện định nghĩa của một phương thức. Chi tiết về các loại được thể hiện trong lớp `TokenTypes`.
- `getText()`: Tên hoặc ký hiệu liên quan tới node.
- `getLineNo()` / `getColumnNo()`: Trả ra số dòng và vị trí trong dòng (đánh số từ 0), phục vụ cho việc tìm lỗi.
- Thông tin quan hệ cây: liên kết tới nút cha, nút con và các nút anh em trực tiếp.

Để làm việc với AST, công cụ cung cấp một cài đặt của interface `FileSetCheck` là `TreeWalker`. Lớp này đóng vai trò:

- Nhận vào một đối tượng `FileText`, đại diện cho nội dung văn bản thô của tệp nguồn.
- Phân tích cú pháp và chuyển mã nguồn thành cây `DetailAST`.
- Duyệt toàn bộ AST và phát sinh các sự kiện tương ứng với từng loại token dựa trên các `Check`.

Thay vì mỗi điều kiện kiểm tra phải tự duyệt cây cú pháp, Checkstyle hỗ trợ cơ chế lọc và phân phối sự kiện thông qua lớp `Check`. Cụ thể:

- Mỗi `Check` đăng ký các loại token mà nó quan tâm.
- Khi `TreeWalker` duyệt đến một `DetailAST` có type phù hợp, nó sẽ gọi các phương thức tương ứng của `Check`.
- Tại đây, `Check` sử dụng thông tin từ `DetailAST` để kiểm tra vi phạm và báo lỗi nếu cần.

2.2.3. Checker

`Checker` là thành phần chịu trách nhiệm quản lý vòng đời của quá trình kiểm tra và duy trì danh sách các module con. Cụ thể, thành phần này đóng vai trò nhận vào mã nguồn yêu cầu kiểm tra, thiết lập môi trường và điều phối dòng dữ liệu tệp tin đến các bộ xử lý thích hợp. Nó không trực tiếp phân tích cú pháp mã nguồn mà ủy quyền cho các module con.

2.2.4. FileSetCheck

Bên cạnh các kiểm tra dựa trên cây cú pháp, Checkstyle còn hỗ trợ các dạng kiểm tra không phụ thuộc vào cấu trúc cú pháp của mã nguồn. Các kiểm tra này được xây dựng dựa trên interface `FileSetCheck`, cho phép xử lý trực tiếp nội dung của tệp nguồn mà không cần thông qua cây `DetailAST`.

Đối với nhóm kiểm tra này, đầu vào chủ yếu là đối tượng `FileText`, đại diện cho toàn bộ nội dung văn bản của một tệp cùng với thông tin dòng và cột. Thay vì phân tích cú pháp, `FileSetCheck` làm việc trực tiếp trên văn bản thô của tệp, thông qua việc duyệt theo dòng hoặc toàn bộ nội dung để phát hiện các vi phạm dựa trên biểu thức chính quy, định dạng (ví dụ như số dòng), hoặc các quy ước ở mức tệp (ví dụ như `HeaderCheck` yêu cầu file phải có header, hay `FileLengthCheck` giới hạn số dòng của một file).

2.2.5. Filter Systems

`Filter` đóng vai trò như các chốt kiểm soát trong luồng sự kiện. Thành phần này sẽ quyết định xem một sự kiện lỗi `AuditEvent` có nên được chuyển tới `AuditListener` để thông báo ra ngoài hay không, từ đó cho phép người dùng bỏ qua một vài lỗi nhất định (ví dụ: `SuppressionFilter` dùng để bỏ qua các cảnh báo). Checkstyle áp dụng chiến lược lọc hai lớp (Two-layer Filtering) can thiệp vào cả đầu vào và đầu ra của quy trình. Điều này đảm bảo tính hiệu quả về hiệu năng và sự linh hoạt trong quản lý kết quả.

- *Before Execution File Filter (Bộ lọc tệp trước thực thi)*

Đây là chốt chặn đầu tiên trong luồng xử lý, hoạt động ngay sau khi `Checker` nhận danh sách tệp nhưng trước khi chuyển giao cho `FileSetCheck` hay `TreeWalker`. Thành phần này đóng vai trò tối ưu hóa hiệu năng bằng cách loại bỏ các tệp không phù hợp khỏi quy trình phân tích cú pháp tốn kém. Thành phần này thường được dùng để loại bỏ các tệp mã nguồn được sinh tự động (generated code), các thư mục tài nguyên, hoặc các tệp không phải Java dựa trên tên hoặc đường dẫn.

- *Audit Filter (Bộ lọc sự kiện)*

Thành phần này hoạt động ở cuối quy trình logic, đóng vai trò như các chốt kiểm soát đối với luồng thông tin đầu ra bằng cách quyết định xem một sự kiện vi phạm (`AuditEvent`) đã được các `Check` phát hiện có nên được chuyển tới `AuditListener` để báo cáo hay không. Thành phần này cho phép người dùng cấu hình các ngoại lệ logic, ví dụ: bỏ qua lỗi dựa trên chú thích `@SuppressWarnings` trong mã nguồn (nhờ `SuppressionFilter`) hoặc lọc lỗi theo mức độ nghiêm trọng (Severity).

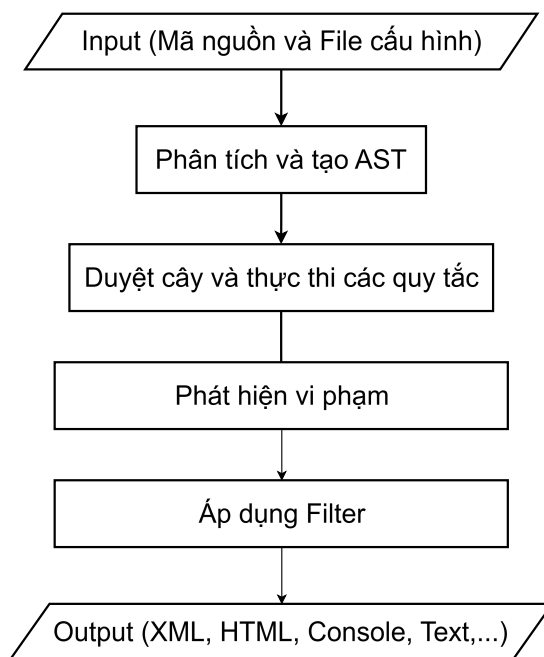
2.2.6. AuditListener

`AuditListener` chịu trách nhiệm lắng nghe kết quả từ quá trình kiểm tra. Từ một các tượng `AuditEvent` chứa thông tin về vị trí và nội dung của lỗi, thành phần này chuyển chúng của hệ thống thành định dạng đầu ra mà người dùng có thể đọc được. Ví dụ: `XMLLogger` xuất ra file XML cho các hệ thống CI/CD, `DefaultLogger` xuất ra Console.

2.3. Nguyên tắc hoạt động

2.3.1. Tổng quan luồng hoạt động

Quá trình vận hành của Checkstyle có thể được khái quát hóa như một pipeline xử lý dữ liệu tuần tự, bắt đầu từ việc khởi tạo môi trường thực thi dựa trên cấu hình người dùng và kết thúc bằng việc xuất báo cáo vi phạm. Trọng tâm của luồng xử lý này là sự phối hợp giữa thành phần điều phối trung tâm **Checker** và các thành phần phân tích chuyên biệt.



Hình 2.1 — Luồng hoạt động của Checkstyle

Khi ứng dụng được khởi chạy, Checkstyle trước tiên sẽ xây dựng một hệ thống phân cấp các module (Module Hierarchy) thông qua cơ chế Reflection, biến các định nghĩa trong tệp cấu hình XML thành các đối tượng Java. Sau khi hệ thống đã sẵn sàng, **Checker** sẽ tiếp nhận danh sách các tệp mã nguồn cần kiểm tra. Tại đây, luồng dữ liệu được chia tách: các tệp tin lần lượt được đưa qua các Filters để loại bỏ những tệp không cần thiết, sau đó được chuyển tiếp đến các **FileSetChecks**.

Trong các bộ kiểm tra này, quá trình phân tích được chia làm hai nhánh chính: phân tích dựa trên văn bản thô và phân tích dựa trên cây cú pháp trừu tượng thông qua **TreeWalker**. Các vi phạm phát hiện được trong quá trình này

sẽ được đóng gói thành các sự kiện và gửi ngược lại cho hệ thống lắng nghe (`AuditListeners`) để định dạng và ghi ra kết quả cuối cùng.

2.3.2. Chi tiết quá trình thực thi

Quá trình thực thi được chia thành các giai đoạn sau đây:

1. Khởi tạo và Xây dựng cấu trúc module

Mọi hoạt động của Checkstyle đều bắt đầu từ `ConfigurationLoader`. Thành phần này chịu trách nhiệm phân tích tệp cấu hình (file `.xml`) và chuyển đổi nó thành một cây đối tượng `Configuration`. Thay vì khởi tạo cứng các lớp, Checkstyle sử dụng mẫu thiết kế Factory kết hợp với Reflection. `PackageObjectFactory` sẽ nhận tên lớp (ví dụ: `"TreeWalker"`, `"AvoidStarImport"`) từ cấu hình, tìm kiếm trong classpath và khởi tạo các đối tượng tương ứng.

Cấu trúc đối tượng được tạo ra phản ánh đúng cấu trúc phân cấp trong tệp XML: `Checker` nằm ở đỉnh, chứa các `FileSetCheck` (như `TreeWalker`) và `AuditListener`. Trong giai đoạn này, các phương thức setter của từng module được gọi tự động để nạp các thuộc tính (properties) tùy chỉnh của người dùng (như độ dài thụt dòng, quy tắc đặt tên) vào trong ngữ cảnh thực thi của từng đối tượng kiểm tra (Check).

2. Sàng lọc và điều phối kiểm tra mã nguồn

Sau khi khởi tạo, quyền điều khiển thuộc về `Checker`. Thành phần này không trực tiếp phân tích mã nguồn mà đóng vai trò điều phối. `Checker` duy trì một danh sách các `FileSetCheck` và các `Filter`.

Trước khi tiến hành kiểm tra mã nguồn, `Checker` thực hiện một bước tối ưu hóa quan trọng đầu tiên là lọc tệp trước thực thi (Before Execution File Filtering). Với mỗi đường dẫn tệp tin trong danh sách đầu vào, `Checker` sẽ đưa nó qua một chuỗi các `BeforeExecutionFileFilter`. Tại đây, các nguyên tắc loại trừ được áp dụng dựa trên tên tệp hoặc đường dẫn (ví dụ: bỏ qua các tệp trong thư mục `target`, tệp cấu hình không phải Java, hoặc mã nguồn được sinh tự động). Nếu một tệp vi phạm một trong các nguyên tắc đã quy định, nó sẽ bị loại bỏ ngay lập tức khỏi luồng xử lý.

Sau khi lọc, với mỗi tệp đầu vào, `Checker` xử lý tiếp bằng cách khởi tạo đối tượng `FileText`. Đối tượng này không chỉ chứa nội dung văn bản thuần túy mà còn xây dựng một ánh xạ từ vị trí kí tự trong file đến dòng/cột tương ứng của nó trong mã nguồn gốc, giúp người dùng dễ dàng xác định vị trí lỗi. Sau đó, `Checker` gọi phương thức `process()` của từng `FileSetCheck` đã đăng ký, chuyển giao đối tượng `FileText` để xử lý tiếp. Cơ chế này tách biệt hoàn toàn việc quản lý danh sách tệp khỏi logic kiểm tra cụ thể.

3. Phân tích cú pháp và Duyệt cây

Đây là giai đoạn phức tạp và quan trọng nhất, nơi phần lớn các quy tắc (Checks) được thực thi. `TreeWalker` chịu trách nhiệm chuyển đổi văn bản thô thành cấu trúc ngữ nghĩa để phân tích.

Đầu tiên, `TreeWalker` sử dụng lớp `JavaParser` (được xây dựng dựa trên ANTLR) để phân tích `FileText`. Quá trình này bao gồm việc Tokenizer (Lexer) chia văn bản thành các token, sau đó Parser sắp xếp chúng thành một AST.

Sau khi có được cấu trúc cây, `TreeWalker` tiến hành duyệt cây theo phương pháp DFS (Depth-First Search) khởi đệ quy. Tại mỗi nút của cây (tương ứng với một cấu trúc ngữ pháp như `METHOD_DEF`, `VARIABLE_DEF`), `TreeWalker` đóng vai trò như một bộ phát sự kiện (Event Dispatcher):

1. Nó kiểm tra xem loại token hiện tại có nằm trong danh sách quan tâm của bất kỳ `Check` nào không. Các `Check` khi khởi tạo phải đăng ký các token mình muốn xử lý thông qua hàm `getDefaultTokens()` hoặc `getRequiredTokens()`.
2. Nếu có, hàm `visitToken()` của các `Check` tương ứng sẽ được gọi tuần tự. Tại đây, các `Check` thực hiện logic kiểm tra cục bộ. Ví dụ, `MethodNameCheck` sẽ kiểm tra xem tên phương thức tại nút hiện tại có khớp với biểu thức chính quy (Regex) đã cấu hình hay không. Các `Check` có thể duy trì trạng thái nội bộ để phân tích các quy tắc cần thông tin tổng hợp từ nhiều node (ví dụ trong `CyclomaticComplexityCheck` hoặc `MethodLengthCheck`, yêu cầu phải lưu các thông tin để có thể tổng hợp vì nó còn phụ thuộc vào thông tin của các nút con).

3. Sau khi toàn bộ cây con của nút hiện tại đã được duyệt xong, `TreeWalker` gọi hàm `leaveToken()`. Đây là thời điểm các `Check` tổng hợp dữ liệu thu được từ cây con để đưa ra quyết định cuối cùng.

Cơ chế này giúp Checkstyle phân tích mã nguồn mà không cần phải duyệt lại cây nhiều lần cho mỗi quy tắc, cũng như không cần phải duyệt lại toàn bộ quy tắc với mỗi nút của cây, từ đó giúp tối ưu hoá hiệu năng đáng kể.

4. Thu thập và Xử lý vi phạm

Trong quá trình thực thi `visitToken` hoặc `leaveToken`, nếu một `Check` phát hiện vi phạm, nó sẽ không in trực tiếp ra màn hình. Thay vào đó, nó gọi phương thức `log()`. Phương thức này tạo ra một đối tượng `AuditEvent` chứa thông tin về tệp, vị trí, thông điệp lỗi và mức độ nghiêm trọng.

Sự kiện này được đẩy ngược lên `TreeWalker`, sau đó chuyển tiếp về `Checker`. Tại đây, `Checker` sử dụng một danh sách các `Filter` để quyết định xem sự kiện này có nên được chấp nhận hay không (ví dụ: `SuppressionFilter` có thể loại bỏ lỗi dựa trên chú thích `@SuppressWarnings`).

Nếu sự kiện vượt qua các bộ lọc, `Checker` sẽ gửi nó đến tất cả các `AuditListener` đã đăng ký. Các Listener này (như `DefaultLogger`, `XMLLogger`) chịu trách nhiệm định dạng dữ liệu sự kiện thành văn bản hoặc XML và ghi ra luồng đầu ra (`OutputStream`) hoặc tệp kết quả.

Việc tách biệt khâu phát hiện lỗi và khâu báo cáo tuân thủ nguyên lý *Single Responsibility*. Nhờ kiến trúc này, Checkstyle có thể thay đổi hoặc mở rộng cơ chế báo cáo để tích hợp với CI/CD hoặc IDE mà không cần sửa đổi logic kiểm tra cốt lõi.

Chương 3

Áp dụng Checkstyle phân tích dự án thực tế

3.1. Lựa chọn dự án và thiết lập môi trường phân tích

3.1.1. Lựa chọn dự án

MegaBasterd là một trình quản lý tải xuống mã nguồn mở được viết bằng ngôn ngữ Java. Nó được thiết kế để đơn giản hóa quá trình tải xuống các tệp lớn từ dịch vụ lưu trữ đám mây Mega.nz.

Các tính năng chính của MegaBasterd bao gồm:

- Hỗ trợ tải xuống hàng loạt từ Mega.nz với tốc độ cao.
- Cung cấp một giao diện dễ sử dụng để quản lý các lần tải xuống.

Dự án có sẵn trên GitHub tại: <https://github.com/tonikelope/megabasterd/>

Lí do lựa chọn dự án MegaBasterd:

- Dự án mã nguồn mở, được viết bằng Java, phù hợp để phân tích bằng Checkstyle.
- Quy mô vừa phải, có đủ số lượng file và dòng code để phân tích.
- Được đóng góp bởi nhiều contributor với thói quen code khác nhau, giúp kiểm thử hiệu quả hơn.

3.1.2. Thiết lập môi trường

Có 3 cách để tích hợp Checkstyle vào quy trình phát triển phần mềm, dự án:

- Sử dụng Checkstyle thông qua command line.
- Tích hợp Checkstyle vào IDE (Eclipse, IntelliJ IDEA).
- Tích hợp Checkstyle vào hệ thống build tự động (Maven, Gradle).

Trong khuôn khổ báo cáo này, nhóm sẽ sử dụng cách thứ nhất – Chạy Checkstyle thông qua command line để thực hiện phân tích mã nguồn dự án MegaBasterd.

Công cụ và phiên bản được sử dụng:

- JDK: 25.0.1
- Checkstyle: 12.1.1
- IDE: IntelliJ IDEA Ultimate 2024.3

Các bước cài đặt Checkstyle:

1. Truy cập trang [Github Release của Checkstyle](#).
2. Tải về file JAR phiên bản mới nhất (tại thời điểm viết báo cáo là 12.1.1).
3. Đặt file JAR vào một thư mục cố định trên máy tính, ví dụ:
`C:\checkstyle\checkstyle-12.1.1-all.jar`.

3.2. Thực thi kiểm thử

Do dự án MegaBasterd là một dự án mã nguồn mở, được đóng góp bởi nhiều contributor với thói quen code khác nhau, nên nhóm quyết định sử dụng bộ quy tắc *Google Checks* có sẵn như một tiêu chuẩn để phân tích mã nguồn dự án này.

Sau khi tải file cấu hình `google_checks.xml` và clone mã nguồn dự án MegaBasterd về máy, tiến hành chạy Checkstyle thông qua command line bằng lệnh:

```
java -jar C:\checkstyle\checkstyle-12.1.1-all.jar \  
-c C:\checkstyle\google_checks.xml \  
-f xml \  
-o C:\checkstyle\checkstyle-result.xml \  
D:\CODE\Java\megabasterd\src
```

Chương trình 3.2 — Lệnh chạy Checkstyle qua command line

Trong đó:

- `-c` : Chỉ định file cấu hình quy tắc kiểm thử.
- `-f xml` : Chỉ định định dạng đầu ra là XML.
- `-o` : Chỉ định file đầu ra để lưu kết quả kiểm thử.

Bên cạnh những tham số trên, Checkstyle CLI còn hỗ trợ nhiều tham số khác để tùy chỉnh quá trình kiểm thử, được mô tả chi tiết trong [tài liệu chính thức của Checkstyle](#).

Người dùng cũng có thể chạy Checkstyle trên một file cụ thể hoặc nhiều file cùng lúc thay vì toàn bộ thư mục, bằng cách thay thế đường dẫn thư mục `D:\CODE\Java\megabasterd\src` trong lệnh trên bằng đường dẫn file cần kiểm tra, ví dụ:

```
D:\CODE\Java\megabasterd\src\...\AboutDialog.java.
```

3.3. Kết quả kiểm thử

Sau khi chạy, Checkstyle sẽ phân tích toàn bộ file có đuôi `.java`, `.properties` và `.xml` (do cấu hình `fileExtensions` của `google_checks`) trong thư mục `src` của dự án MegaBasterd, và ghi kết quả kiểm thử vào file `checkstyle-result.xml`. Dưới đây là một phần của file kết quả sau khi thực thi kiểm thử:

```

<?xml version="1.0" encoding="UTF-8"?>
<checkstyle version="12.1.1">
<file name="D:\CODE\Java\megabasterd\src\...\AboutDialog.java">
<error line="10" column="1" severity="warning" message="'package' should be separated
from previous line."
source="com.puppycrawl.tools.checkstyle.checks.whitespace.EmptyLineSeparatorCheck"/>
<error line="12" column="51" severity="warning" message="Using the '.*' form of import
should be avoided - com.tonikelope.megabasterd.MainPanel.*."
source="com.puppycrawl.tools.checkstyle.checks.imports.AvoidStarImportCheck"/>
...
</file>
<file name="D:\CODE\Java\megabasterd\src\...\ChunkDownloader.java">
...
<error line="28" severity="warning" message="Summary javadoc is missing."
source="com.puppycrawl.tools.checkstyle.checks.javadoc.SummaryJavadocCheck"/>
...
<error line="235" column="5" severity="warning" message="'method def rcurly' has
incorrect indentation level 4, expected level should be 2."
source="com.puppycrawl.tools.checkstyle.checks.indentation.IndentationCheck"/>
</file>
<file name="D:\CODE\Java\megabasterd\src\...\MegaProxyServer.java">
...
<error line="72" column="37" severity="warning" message="Empty catch block."
source="com.puppycrawl.tools.checkstyle.checks.blocks.EmptyCatchBlockCheck"/>
...
<error line="93" severity="warning" message="Line is longer than 100 characters (found
129)." source="com.puppycrawl.tools.checkstyle.checks.sizes.LineLengthCheck"/>
...
</file>
<file name="D:\CODE\Java\megabasterd\src\...\UploadView.java">
<error line="16" column="1" severity="warning" message="Import statement for
'java.lang.Integer.MAX_VALUE' is in the wrong order. Should be in the 'STATIC' group,
expecting not assigned imports on this line."
source="com.puppycrawl.tools.checkstyle.checks.imports.CustomImportOrderCheck"/>
<error line="428" column="5" severity="warning" message="WhitespaceAround: '}' is not
followed by whitespace. Empty blocks may only be represented as {} when not part of a
multi-block statement (4.1.3)"
source="com.puppycrawl.tools.checkstyle.checks.whitespace.WhitespaceAroundCheck"/>
<error line="552" column="17" severity="warning" message="Abbreviation in name
'updateCBC' must contain no more than '1' consecutive capital letters."
source="com.puppycrawl.tools.checkstyle.checks.naming.AbbreviationAsWordInNameCheck"/>
...
</file>
</checkstyle>

```

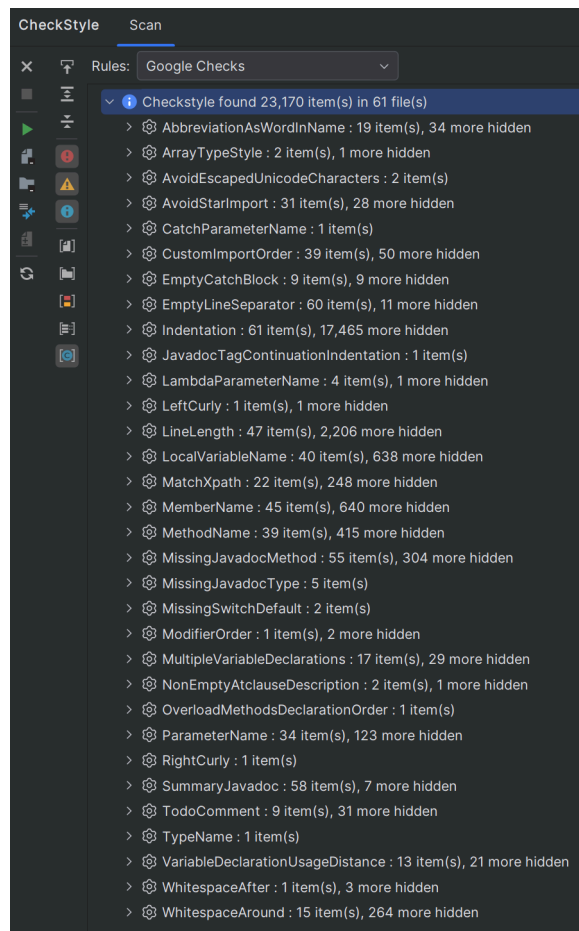
Chương trình 3.3 — Kết quả phân tích mã nguồn dự án MegaBasterd

3.4. Phân tích kết quả

Sau khi phân tích file `checkstyle-result.xml`, ta thấy rằng hầu hết các lỗi vi phạm thuộc dạng *Indentation* (thụt lề) vì dự án MegaBasterd sử dụng 4 space cho mỗi cấp thụt lề, trong khi *Google Checks* yêu cầu 2 space. Ngoài ra còn có một số lỗi khác như:

- *LineLength*: Độ dài dòng vượt quá 100 ký tự.
- *WhitespaceAround*: Thiếu khoảng trắng xung quanh các toán tử.
- *AvoidStarImport*: Sử dụng `import` dạng `.*`.
- *EmptyCatchBlock*: Khối `catch` trống.
- *SummaryJavadoc*: Thiếu JavaDoc tóm tắt cho class hoặc method
- Các lỗi tên biến, tên phương thức không tuân thủ quy ước đặt tên của Google.

Để tổng quát hơn, ta phân tích kèm với Plugin Checkstyle-IDEA trên IntelliJ IDEA:



Hình 3.1 — Tổng quan các lỗi vi phạm trong dự án MegaBasterd

Có thể thấy, trong dự án MegaBasterd có tổng cộng 23170 vi phạm trong 61/61 file mã nguồn. Trong đó, lỗi phổ biến nhất vẫn là *Indentation* (17526 vi phạm trong 61/61 file), theo sau là *Linelenhth* (2253 vi phạm trong 47/61 file) và *MemberName* (685 vi phạm trong 35/61 file),...

Điều này cho thấy dự án MegaBasterd không sử dụng Code Convention của Google, dẫn đến việc vi phạm nhiều quy tắc kiểm thử của Google Checks.

3.5. Đề xuất khắc phục

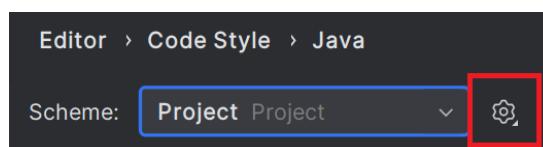
Để giảm thiểu các lỗi vi phạm được phát hiện bởi Checkstyle, có thể chỉnh sửa file cấu hình `google_checks.xml` để bỏ qua một số quy tắc không phù hợp với dự án, ví dụ như *Indentation* hoặc *LineLength* và giữ lại những quy tắc quan trọng. Hoặc ta có thể format lại mã nguồn dự án để tuân thủ các quy tắc của Google Checks.

Có 2 cách để format lại mã nguồn dự án:

- Sử dụng tính năng Reformat Code có sẵn trong IntelliJ IDEA kèm với file cấu hình [Google Java Style](#).
- Sử dụng plugin [google-java-format](#) để tự động format mã nguồn theo chuẩn Google.

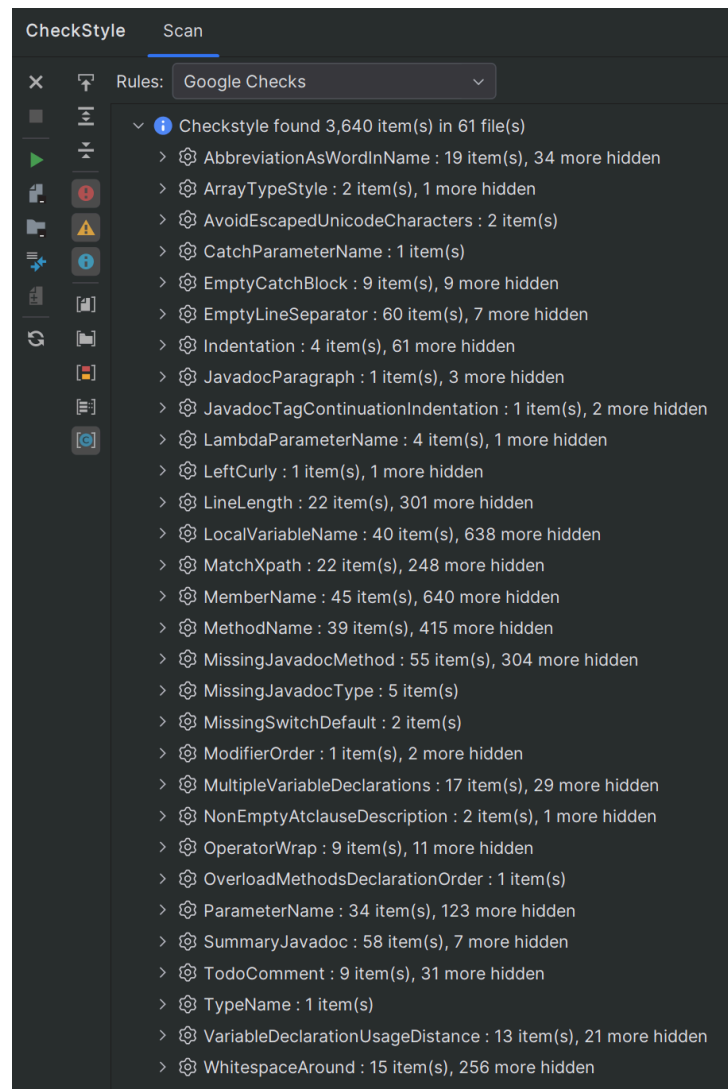
Nhóm sẽ thử áp dụng cách thứ nhất để format lại mã nguồn dự án MegaBasterd và chạy lại Checkstyle để kiểm tra kết quả.

1. Tải file cấu hình [intellij-java-google-style.xml](#).
2. Trong IntelliJ IDEA, vào **File** → **Settings** → **Editor** → **Code Style** → **Java**.
3. Nhấn vào biểu tượng bánh răng và chọn **Import Scheme** → **IntelliJ IDEA code style XML**.



4. Chọn file cấu hình đã tải về và nhấn **OK**.
5. Mở dự án MegaBasterd và sử dụng tính năng Reformat Code để tự động định dạng lại mã nguồn theo chuẩn Google.

Sau khi format lại mã nguồn và chạy lại Checkstyle, thu được [file kết quả](#). Sử dụng Plugin Checkstyle-IDEA để tổng quan hóa kết quả:



Có thể thấy số lượng vi phạm đã giảm đáng kể từ 23170 xuống còn 3640 vi phạm, trong đó lỗi *Indentation* đã không còn xuất hiện nữa. Thay vào đó là các lỗi như *MemberName* (685 vi phạm), *LocalVariableName* (678 vi phạm), *MethodName* (454 vi phạm),...

Kiểm tra nhanh file `APIException.java` vi phạm lỗi *MemberName*:

```
...
public abstract class APIException extends Exception {

    protected Integer _code;

    ...
}
```

Tên biến `_code` không tuân thủ quy ước đặt tên của Google (sử dụng dấu gạch dưới ở đầu tên biến), có thể ý đồ của tác giả là để biểu thị đây là biến protected. Để khắc phục lỗi này, có thể đổi tên biến thành `code`.

Chương 4

So sánh

4.1. Đánh giá

Để thấy rõ hơn vị trí và vai trò của Checkstyle trong quy trình phát triển phần mềm, cần đặt công cụ này trong mối tương quan với các giải pháp phân tích mã nguồn phổ biến khác. Trong đó, SonarQube là một đại diện tiêu biểu nhờ khả năng phân tích chất lượng mã nguồn một cách toàn diện ở cấp độ dự án. Phần đánh giá dưới đây sẽ tập trung so sánh Checkstyle và SonarQube dựa trên các tiêu chí như mục đích sử dụng, phạm vi kiểm tra, cách thức triển khai và hiệu quả áp dụng trong thực tế, từ đó làm rõ ưu điểm, hạn chế cũng như bối cảnh phù hợp của từng công cụ.

Bảng 4.1 — So sánh Checkstyle và SonarQube

Tiêu chí	Checkstyle	SonarQube
Mục đích	Chủ yếu kiểm tra phong cách và quy ước mã nguồn.	Kiểm tra chất lượng toàn diện của code, bao gồm lỗi, lỗ hổng bảo mật, khả năng bảo trì, độ bao phủ của bộ kiểm thử.
Phạm vi	Kiểm tra từng file riêng biệt.	Kiểm tra toàn dự án như một thể thống nhất.
Tiêu chí	Cách code được viết: naming convention, indentation, whitespaces, bracket placement, ...	Tập trung vào hành vi và chất lượng thực thi của mã nguồn; phong cách viết mã không phải trọng tâm chính.
Ngôn ngữ hỗ trợ	Java	Nhiều ngôn ngữ
Cách dùng	Chỉ cần một file cấu hình XML, dùng như app CLI standalone, hoặc tích hợp vào IDE & build tool dưới dạng plugin.	Cần server tập trung, dự án phải được cấu hình trên server.
Độ phức tạp	Nhẹ và nhanh, trả về kết quả kiểm thử ngay lập tức.	Triển khai phức tạp hơn, yêu cầu hạ tầng server và thời gian phân tích dài hơn.
Áp dụng thực tế	Chạy mỗi lần build	Chạy trong CI/CD pipeline hoặc mỗi pull request
Chi phí	Miễn phí và mã nguồn mở	Có bản miễn phí và bản trả phí nhiều tính năng hơn

Từ bảng so sánh trên, có thể thấy SonarQube là một công cụ mạnh mẽ hơn so với Checkstyle nhờ khả năng kiểm tra toàn diện của mình. Sở dĩ SonarQube có thể phân tích toàn bộ dự án là nhờ kiến trúc Client-Server. Cụ thể hơn, SonarScanner đóng vai trò client sẽ đọc nội dung của toàn bộ các file mã nguồn và gửi chúng cho server. Server, sau khi thu thập đầy đủ dữ liệu, sẽ tiến hành xử lý và phân tích toàn bộ mã nguồn nhằm hiểu rõ cấu trúc, thành phần và mối quan hệ giữa chúng. Từ đó, SonarQube có thể thực thi các kiểm thử liên file phức tạp.

Dù không có các tính năng nâng cao như SonarQube, Checkstyle vẫn có một vài ưu điểm rõ rệt: Miễn phí, nhẹ, nhanh và dễ sử dụng. Với thời gian chạy được tính bằng giây, trong thực tế, lập trình viên có thể chạy Checkstyle để đảm bảo code convention ở mọi lần build, và quản lí dự án có thể đặt Checkstyle ở những giai đoạn đầu trong CI/CD pipeline để loại bỏ sớm các bản code không tuân thủ theo quy tắc trước khi chúng đến được các giai đoạn tốn thời gian, tốn tài nguyên hơn. Vì vậy, Checkstyle vẫn có giá trị rất lớn trong quy trình phát triển phần mềm hiện nay.

4.2. Kết luận

Tóm lại, Checkstyle là một công cụ phân tích mã tĩnh hiệu quả, tập trung vào việc đảm bảo mã nguồn Java tuân thủ các quy ước và tiêu chuẩn mã hóa đã được định nghĩa. Với cơ chế hoạt động đơn giản, cấu hình linh hoạt thông qua tệp XML và thời gian chạy nhanh, Checkstyle giúp lập trình viên phát hiện sớm các vi phạm về phong cách code ngay trong quá trình phát triển.

Nhờ đặc tính nhẹ, dễ tích hợp và là một dự án mã nguồn mở miễn phí, Checkstyle đặc biệt phù hợp để sử dụng thường xuyên trong quá trình build hoặc ở các giai đoạn đầu của CI/CD pipeline nhằm loại bỏ sớm các đoạn mã không tuân thủ quy tắc. Điều này góp phần nâng cao tính nhất quán, khả năng đọc hiểu và khả năng bảo trì của mã nguồn. Mặc dù không hướng tới việc phân tích logic hay hành vi thực thi của chương trình, Checkstyle vẫn giữ vai trò quan trọng trong quy trình phát triển phần mềm hiện đại như một công cụ kiểm soát chất lượng mã nguồn ở mức cơ bản nhưng thiết yếu.

Tài liệu tham khảo

- [Checkstyle - Java code quality tool](#)
- [SonarQube](#)